DOCAPOSTE Institute

FORMATION · 1 JOUR · CLASSE VIRTUELLE

GitHub Copilot niveau avancé

Guide des 6 TP individuels · fil rouge SalleLibre
Instructions, skills, agents personnalisés et orchestration · 6 cheat sheets en annexe

Houssem Eddine LASSOUED · Docaposte Institute

Version du 12 novembre 2026 · formats vérifiés sur code.visualstudio.com et docs.github.com le 11 novembre 2026

Mode d’emploi

Pendant une journée, vous construisez **SalleLibre**, une API REST fictive de réservation de salles de réunion (salles, réservations sans chevauchement, authentification JWT et rôles, recherche de salles libres, documentation OpenAPI), avec GitHub Copilot dans VS Code. Stack au choix : TypeScript · Express, Python · FastAPI, Java · Spring Boot ou C# · ASP.NET Core.

Les six TP s’enchaînent sur **le même dépôt**. Vous connaissez déjà Copilot : on ne revient pas sur les bases. Chaque TP ajoute une brique à votre dossier `.github`, du fichier d’instructions jusqu’à une équipe d’agents orchestrée.

Les TP sont **individuels** : chacun sur son poste, minuteur affiché, points d’étape oraux toutes les 8 à 10 minutes. Bloqué plus de 5 minutes ? Dites-le au micro : partage d’écran de 2 minutes. Les textes à copier (prompts, squelettes de fichiers) sont collés dans le Chat de la classe virtuelle.

Les 6 TP

TP	Module	Durée	Ce que vous ajoutez
1 · Prompts mesurés	M1 · Prompt engineering	20 min	une grille de mesure, `overlaps`
2 · API REST complète	M2 · Architectures	30 min	`PLAN.md`, `.github/copilot-instructions.md`, l’API
3 · Suite de tests	M3 · Tests	25 min	`tests.instructions.md`, la skill `matrice-de-tests`
4 · Refactoring	M4 · Refactoring	25 min	l’agent `Refactoreur` et sa skill `refactoring-sur`
5 · Audit de sécurité	M5 · Sécurité	25 min	l’agent `Auditeur` (sous-agent), `security.instructions.md`
6 · Feature orchestrée	M6 · Équipe	25 min	l’agent `Chef d’orchestre` et ses équipiers, une PR

Votre dossier .github en fin de journée

```
.github/
├── copilot-instructions.md          TP 2 · les règles de la maison (≤ 40 lignes)
├── instructions/
│   ├── tests.instructions.md        TP 3 · applyTo: "tests/**"
│   └── security.instructions.md     TP 5 · applyTo: "src/api/**,src/services/**"
├── skills/
│   ├── chevauchement/SKILL.md       TP 2 bonus · /chevauchement
│   ├── matrice-de-tests/            TP 3 · SKILL.md + CASES.modele.md
│   └── refactoring-sur/SKILL.md     TP 4
└── agents/
    ├── refactoreur.agent.md         TP 4
    ├── auditeur.agent.md            TP 5 · lecture seule, sous-agent
    ├── architecte.agent.md          TP 6 · fourni
    ├── testeur.agent.md             TP 6 · fourni
    ├── documentaliste.agent.md      TP 6 · fourni
    └── chef-orchestre.agent.md      TP 6
```

Règles du jeu

- **Session Copilot**, permissions **manuelles** : vous approuvez chaque commande de l’agent.
- **Aucune donnée réelle** dans un prompt : ni code client, ni secret, ni donnée personnelle. SalleLibre est fait pour ça.
- **Un lot, une preuve, un commit.** Pas d’étape suivante sans test vert.
- **Le diff des tests se relit avant le diff du code.** Un test supprimé, sauté ou affaibli annule le « c’est fini ».
- **Une règle ignorée se corrige dans le fichier**, pas en la répétant dans le prompt. Vérifiez la liste « Références » sous chaque réponse.
- **En retard ?** Passez au TP suivant sur votre code tel quel : chaque TP fonctionne sur une API partielle. La méthode compte plus que l’exhaustivité.

Avant la formation

- VS Code à jour, extension GitHub Copilot connectée (Chat inclus), licence Copilot active.
- Dans le Chat : les sélecteurs **Session** (Copilot), **rôle** (Ask, Plan, Agent) et **modèle** sont visibles.
- Le runtime de votre stack (Node 22, Python 3.12, JDK 21 ou .NET 8+) et son framework de test (Vitest/Jest, pytest, JUnit 5, xUnit).
- Un dépôt Git vide `sallelibre`, premier commit poussé.
- Classe virtuelle : micro, caméra si possible, idéalement deux écrans (la formation d'un côté, VS Code de l'autre).

Sommaire

TP 1	Réécrire des prompts basiques et mesurer
TP 2	Générer une API REST complète, guidée par vos instructions
TP 3	Une suite de tests complète, avec votre première skill
TP 4	Refactoring en profondeur avec votre agent Refactoreur
TP 5	Audit de sécurité par un agent Auditeur
TP 6	Projet de clôture : une feature orchestrée
A	Session, Chat et raccourcis
B	Prompts avancés : la boîte à outils
C	Les fichiers qui guident Copilot
D	Agents et orchestration
E	Tests et refactoring
F	Sécurité, dépendances et revue

MES NOTES

TP 1

MODULE 1 · PROMPT ENGINEERING AVANCÉ

Réécrire des prompts basiques et mesurer

⏱ 20 min | Matin · vers 9:50 | Individuel

`prompts/tp1-mesures.md`, `src/domain/overlap.*`, `prompts/meilleur-prompt.md`

OBJECTIF

Prouver, chiffres à l'appui, qu'un prompt structuré produit un meilleur code qu'un prompt basique sur trois tâches de SalleLibre. Mesurer, pas ressentir.

Point de départ

- Le dépôt `sallelibre`, vide, dans la stack de votre choix.
- Réglages : **session Copilot**, **rôle Agent**, et **un seul modèle** choisi dans la liste pour toutes les mesures (pas Auto : sinon vous comparez des modèles, pas des prompts).

Les trois prompts de départ

- P1 · « fais une fonction qui vérifie si deux réservations se chevauchent »
- P2 · « génère les modèles de données de l'appli »
- P3 · « écris la fonction qui trouve les salles libres »

Étapes

1. Créez `prompts/tp1-mesures.md` et fixez votre grille **avant** le premier essai : compile ? tests passent ? bornes correctes (10h–11h puis 11h–12h : pas de chevauchement) ? nombre d'allers-retours.
2. Lancez chaque prompt tel quel **deux fois**, chaque fois dans une **nouvelle session** (Ctrl+N dans la vue Chat). Notez la grille.
3. Réécrivez chaque prompt avec les cinq composants : contexte, tâche bornée, contraintes, exemples attendus, critère de fin (voir l'annexe B).
4. Relancez chaque version V2 deux fois en session neuve. Notez.
5. Gardez le code du meilleur résultat pour `overlaps` : c'est la première brique de SalleLibre.
6. Copiez votre meilleur prompt dans `prompts/meilleur-prompt.md` : il deviendra une skill au TP 2.

C'EST TERMINÉ QUAND…

- [ ] Session Copilot, rôle Agent, un seul modèle pour toutes les mesures
- [ ] Trois prompts réécrits avec les cinq composants
- [ ] Chaque version lancée 2 fois en session neuve, grille remplie
- [ ] `overlaps` et ses tests commités, meilleur prompt conservé

PIÈGES À OBSERVER

- Changer deux choses à la fois entre V1 et V2 : vous ne savez plus ce qui a fait la différence.
- Rester dans la même session : l'historique des essais précédents devient un contexte.
- Juger « à l'œil » : sans grille décidée avant, on retient le résultat qui nous arrange.

BONUS

Relancez votre meilleur prompt avec un autre modèle : la grille bouge-t-elle plus d'un modèle à l'autre que d'un prompt à l'autre ?

COINCÉ ?

- Les bornes sont toujours fausses ? Donnez l'exemple qui tranche dans le prompt : `[10:00, 11:00[` et `[11:00, 12:00[` ne se chevauchent pas.
- Le code part dans tous les sens ? Ajoutez le critère de fin : « Arrête-toi quand ces 4 exemples sont des tests verts. »

TP 2

MODULE 2 · MODULES & ARCHITECTURES

Générer une API REST complète, guidée par vos instructions

⏱ 30 min | Matin · vers 11:05 | Individuel

`PLAN.md` , `.github/copilot-instructions.md` , `src/**` , `openapi.yaml`

OBJECTIF

L'API SalleLibre fonctionne, guidée par **vos** règles de la maison écrites une fois : authentification JWT, CRUD des salles (admin), réservations sans chevauchement, validation, documentation OpenAPI.

Point de départ

- Votre dépôt après le TP 1 : `overlaps` et ses tests.
- Persistance **en mémoire** : pas de base de données aujourd'hui, les tests restent rapides.

Étapes

1. **Rôle Plan** : demandez l'arborescence (le rôle de chaque dossier en une ligne) et la liste des interfaces à créer en premier. Aucune implémentation. Validez, puis enregistrez le plan dans `PLAN.md` .
2. Écrivez `.github/copilot-instructions.md` , **40 lignes au plus** : la stack, les couches de `PLAN.md` , les erreurs métier et leurs codes HTTP, la règle des dépendances. `/init` dans une session d'agent peut proposer un premier jet : raccourcissez-le.
3. **Rôle Agent, lot par lot** : domaine (reprenez `overlaps`), repositories en mémoire, services. Un lot, ses tests, un commit.
4. Routes, validation, authentification, puis la documentation OpenAPI.
5. À chaque réponse, dépliez **Références** : votre fichier d'instructions doit y figurer. Une règle ignorée ? Corrigez le fichier, pas le prompt.

Modèle à copier

```
# SalleLibre · instructions pour Copilot
API REST FICTIVE de réservation de salles.
TypeScript, Express, Node.js 22 (adaptez à votre stack).

## Architecture (voir PLAN.md)
- domain/ pur : ni HTTP, ni stockage.
- Routes fines → services (règles) → repositories (interfaces).
- Erreur métier typée : ConflictError → 409, ForbiddenError → 403,
  jamais une erreur générique.

## Règles maison
- Aucune dépendance ajoutée sans mon accord.
- Toute entrée validée avant le service.
- Aucun secret dans le code : variables d'environnement.

## Tests
- Un test par règle métier ; ne jamais modifier tests/acceptance/.
```

C'EST TERMINÉ QUAND…

- [ ] Plan validé dans `PLAN.md` avant toute ligne de code
- [ ] `copilot-instructions.md` (≤ 40 lignes) cité dans les Références
- [ ] 409 sur chevauchement, 401 sans jeton, 403 sans le bon rôle
- [ ] Documentation OpenAPI accessible et à jour

PIÈGES À OBSERVER

- Tout générer en un seul prompt : un diff de 30 fichiers que personne ne relira.
- Un fichier d'instructions encyclopédique : chaque ligne se paie à chaque requête, et les règles utiles s'y noient.
- `copilot-instructions.md` **et** `AGENTS.md` qui se contredisent : Copilot combine les deux, sans priorité.

BONUS

Transformez votre meilleur prompt du TP 1 en skill : `.github/skills/chevauchement/SKILL.md` , avec un en-tête `name: chevauchement` , une `description` et `disable-model-invocation: true` , puis votre prompt. Tapez `/chevauchement` dans le Chat.

COINCÉ ?

- Les instructions n'apparaissent pas dans les Références ? Vérifiez le chemin exact (`.github/copilot-instructions.md`) et ouvrez **Chat: Open Customizations**.

TP 3

MODULE 3 · AUTOMATISER LES TESTS

Une suite de tests complète, avec votre première skill

⏱ 25 min | Matin · vers 12:05 | Individuel

`tests.instructions.md` , `.github/skills/matrice-de-tests/` , `tests/**`

OBJECTIF

Écrire votre méthode de test **une fois**, dans une skill, puis l'appliquer à l'API du TP 2 : matrice de cas, tests unitaires paramétrés, doubles de test, scénarios d'intégration, couverture.

Point de départ

- L'API du TP 2 et son `copilot-instructions.md`.
- Le framework de test de votre stack, prêt.

Étapes

1. Créez `.github/instructions/tests.instructions.md` avec `applyTo: "tests/**"` : framework, nommage, Arrange / Act / Assert, horloge fixe, ne jamais modifier `tests/acceptance/`.
2. Créez la skill `.github/skills/matrice-de-tests/` : un `SKILL.md` dont la **description** dit quand l'utiliser, la méthode en quatre colonnes, et un modèle `CASES.modele.md` à côté.
3. Demandez « Prépare les tests de BookingService » **sans citer la skill**. Elle doit se charger seule : vérifiez les Références. Si elle ne se charge pas, c'est sa description qui est mauvaise.
4. Validez la matrice (`tests/CASES.md`) : tranchez chaque « ? » avant toute génération.
5. Tests unitaires paramétrés du domaine et des services, doubles injectés aux frontières (persistance, horloge).
6. Trois scénarios d'intégration, dont un **403** (supprimer la réservation d'un autre).
7. Couverture : ciblez les branches manquantes. Puis relisez **le diff des tests** avant de dire « fini ».

Modèle à copier

```
---
name: matrice-de-tests
description: Préparer ou compléter les tests de SalleLibre à partir
d'une
  matrice de cas (nominal, limites, négatifs, sécurité), avant tout
code de test.
---
# Matrice de tests
1. Lis la règle métier et le code concerné.
2. Remplis le modèle [CASES.modele.md](./CASES.modele.md) : 4
colonnes,
   au moins 2 cas par colonne. Marque « ? » ce qui n'est pas tranché.
3. ARRÊTE-TOI : la matrice doit être validée avant tout test.
4. Un test paramétré par règle, une ligne de table par cas.
5. Ne supprime, ne saute et n'affaiblis jamais un test existant.
```

C'EST TERMINÉ QUAND…

- [ ] La skill se charge seule (visible dans les Références)
- [ ] Matrice en 4 colonnes validée avant génération
- [ ] Trois scénarios d'intégration, dont un 403
- [ ] Diff des tests relu, services couverts à 80 % des branches

PIÈGES À OBSERVER

- Une description de skill vague (« aide pour les tests ») : elle ne se charge jamais, ou partout.
- `applyTo: "test/**"` au lieu de `"tests/**"` : le fichier n'est jamais appliqué, rien ne vous prévient.
- Des tests « verts » qui ne vérifient rien : assertion faible, vraie date du jour, mock du domaine.
- L'agent qui « fait passer » la suite en modifiant un test.

BONUS

Changez un `<` en `<=` dans `overlaps` : au moins un test doit casser. Sinon, ajoutez la règle à la skill.

COINCÉ ?

- La skill ne se charge pas ? Tapez `/matrice-de-tests` pour l'appeler à la main, puis réécrivez sa description avec les mots de votre demande.
- Le test 403 échoue ? Normal si l'API ne vérifie pas le propriétaire : gardez-le, c'est la faille du TP 5.

TP 4

MODULE 4 · OPTIMISATION & REFACTORING

Refactoring en profondeur avec votre agent Refactoreur

⏱ 25 min | Après-midi · vers 14:15 | Individuel

`REFACTO.md` , `.github/agents/refactoreur.agent.md` , `.github/skills/refactoring-sur/`

OBJECTIF

Optimiser le code du matin (SOLID, patterns, performance) avec **votre** agent personnalisé, sans changer le comportement : chaque refactoring prouvé par des tests inchangés.

Point de départ

- L'API testée du TP 3, tous les tests au vert, tout commité.

Étapes

1. **Rôle Ask** : anti-patterns et violations SOLID de votre code, dans `REFACTO.md`. Un rapport, aucune modification. Priorisez trois refactorings.
2. Écrivez la skill `.github/skills/refactoring-sur/SKILL.md` : vos gestes de refactoring en dix lignes.
3. Écrivez l'agent `.github/agents/refactoreur.agent.md` : quatre outils au plus, des règles vérifiables.
4. Choisissez **Refactoreur** dans la liste des rôles : un refactoring, les tests, un commit `refactor : …`. Trois fois.
5. Optimisez `findAvailableRooms` : mesurez avant (`bench/availability.*`), donnez la mesure et la cible à l'agent, mesurez après.

Modèle à copier

```
---
name: Refactoreur
description: Refactore SalleLibre par petits pas sûrs, sans changer le
  comportement. Ne corrige pas de bug, ne touche pas aux tests.
tools: ['read', 'search', 'edit', 'execute']
skills: ['refactoring-sur']
---
Tu es le Refactoreur de SalleLibre.
- Lance les tests avant et après chaque geste.
- Un refactoring à la fois ; test rouge : annule le geste.
- Aucune correction de bug pendant un refactoring : signale-le.
- Ne modifie jamais tests/.
- Termine par le diff résumé et le message de commit proposé.
```

C'EST TERMINÉ QUAND…

- ☐ Rapport d'anti-patterns en rôle Ask, priorisé
- ☐ Refactoreur : 4 outils au plus, règles vérifiables
- ☐ Trois refactorings, un commit chacun, aucun fichier de test modifié
- ☐ Benchmark avant / après de la recherche de salles

PIÈGES À OBSERVER

- « Applique SOLID » : un diff géant. Nommez le symptôme, le principe et la cible, un refactoring par demande.
- Des tests modifiés dans le même diff : ils ne prouvent plus que le comportement est préservé.
- `tools: ['*']` par confort : l'agent peut tout faire, y compris ce que vous n'aviez pas prévu.
- Optimiser sans mesurer : l'algorithme « plus malin » est parfois plus lent sur vos volumes.

BONUS

Retirez `execute` de l'agent et relancez : que fait-il quand il ne peut plus lancer les tests ?

COINCÉ ?

- Refactoreur n'apparaît pas dans la liste ? Vérifiez l'extension `.agent.md` , le dossier `.github/agents/` , puis **Chat: Open Customizations › Agents**.
- La skill n'est pas chargée ? Le nom dans `skills:` doit être exactement celui du dossier.

TP 5

MODULE 5 · SÉCURITÉ & AUDIT

Audit de sécurité par un agent Auditeur

⏱ 25 min | Après-midi · vers 15:05 | Individuel

`.github/agents/auditeur.agent.md` , `SECURITY-AUDIT.md` , `tests/security/**` , `security.instructions.md`

OBJECTIF

Faire auditer le code de SalleLibre par un agent **qui ne peut rien modifier**, sollicité comme sous-agent, puis prouver et corriger chaque faille avec un test et un prompt ciblé.

Point de départ

- L'API refactorée du TP 4, tests au vert.
- Le test d'intégration 403 du TP 3, s'il échoue encore : c'est votre première faille.

Étapes

1. Écrivez `.github/agents/auditeur.agent.md` : `read` et `search` seulement, `user-invocable: false` , la grille OWASP et le format du rapport dans le corps.
2. En **rôle Agent** : « Utilise l'agent Auditeur comme sous-agent pour auditer src/api. » Dépliez l'appel replié dans le Chat : le sous-agent a travaillé dans son propre contexte et ne renvoie que son rapport.
3. Classez les failles dans `SECURITY-AUDIT.md` : criticité, preuve, correctif proposé.
4. Pour chaque faille : un test qui l'exploite (**rouge**), un correctif ciblé, **vert**. Commencez par le DELETE d'autrui.
5. Rendez les règles permanentes : `.github/instructions/security.instructions.md` avec `applyTo: "src/api/**,src/services/**"` .

Modèle à copier

```
---
name: Auditeur
description: Audite un diff ou un dossier de SalleLibre selon l'OWASP
  Top 10 : 2025. Lecture seule : rapporte, ne corrige jamais.
tools: ['read', 'search']
user-invocable: false
---
Tu es l'Auditeur sécurité de SalleLibre.
- Grille : OWASP Top 10 : 2025 (contrôle d'accès d'abord).
- Pour chaque faille : fichier, ligne, criticité,
  la requête HTTP qui l'exploite, le correctif proposé.
- Classe du plus critique au moins critique.
- Tu ne modifies aucun fichier.
```

C'EST TERMINÉ QUAND…

- [ ] Auditeur : `read` et `search` seulement, sollicité comme sous-agent
- [ ] Au moins quatre failles trouvées et classées
- [ ] Un test d'exploitation par faille corrigée, dont le DELETE d'autrui (403)
- [ ] `security.instructions.md` commité

PIÈGES À OBSERVER

- Corriger sans test : la faille reviendra au prochain refactoring.
- Un auditeur qui a `edit` : il « corrige » en silence, et vous perdez la preuve.
- Croire l'audit exhaustif : l'IA relit avec intelligence mais sans garantie. Les scanners restent nécessaires.
- Fermer une alerte « faux positif » sans le prouver.

BONUS

Lancez CodeQL ou Semgrep et comparez avec le rapport de l'Auditeur : qui a trouvé quoi ?

COINCÉ ?

- L'agent principal ne délègue pas ? Nommez l'agent exactement (« Auditeur », sensible à la casse) et demandez explicitement un sous-agent.
- L'API semble propre ? Testez : DELETE de la réservation d'un autre utilisateur, réponse qui expose le hash du mot de passe, secret JWT en dur, absence de limite de débit sur `/login` .

TP
6

MODULE 6 · COPILOT EN ÉQUIPE

Projet de clôture : une feature orchestrée

⏱ 25 min | Après-midi · vers 16:15 | Individuel

`.github/agents/chef-orchestre.agent.md` (+ architecte, testeur, documentaliste), une PR

OBJECTIF

Livrer une feature complète de SalleLibre (code, tests, documentation, sécurité) en orchestrant vos agents, et garder la main aux deux décisions : **le plan**, puis **le diff**. Deux ou trois volontaires présentent ensuite leur travail (4 minutes).

Point de départ

- Votre dépôt du TP 5, avec `refactoreur.agent.md` et `auditeur.agent.md`.
- Les agents Architecte, Testeur et Documentaliste : collés dans le Chat (annexe D).
- Une feature au choix : **réservations récurrentes** (chaque mardi, avec exceptions), **liste d'attente** ou **export iCal**.

Étapes

1. Copiez les trois agents fournis dans `.github/agents/`.
2. Écrivez `chef-orchestre.agent.md` : l'outil `agent`, la délégation par nom exact, et **deux points d'arrêt** (le plan, le diff).
3. Choisissez **Chef d'orchestre** dans la liste des rôles et décrivez la feature.
4. Quand il s'arrête : relisez le plan, corrigez-le si besoin, validez.
5. Suivez les sous-agents dans le Chat (appels repliés) : tests rouges, code, audit, documentation.
6. À la fin : relisez le diff **des tests** d'abord, puis celui du code. Ouvrez la PR avec une description.
7. Volontaire pour présenter ? 4 min : démo avec un sous-agent déplié, preuves, un refus argumenté.

Modèle à copier

```
---
name: Chef d'orchestre
description: Mène une feature de SalleLibre de bout en bout en
  déléguant aux agents spécialisés.
tools: ['agent', 'read', 'search', 'edit', 'execute', 'todo']
---
Tu coordonnes. Sollicite chaque agent comme sous-agent, par son nom
exact.
1. Architecte : un plan en lots.
   ARRÊTE-TOI : j'approuve le plan.
2. Pour chaque lot :
   a. Testeur : les tests du lot, ROUGES.
   b. Toi : le code minimal, jusqu'au vert. Ne modifie aucun test.
   c. Auditeur : audit du diff. Faille bloquante : retour en a.
3. Documentaliste : OpenAPI et README.
4. Résume lots, tests, failles, risques restants.
   Je relis le diff avant la PR.
```

C'EST TERMINÉ QUAND…

- [ ] Délégation par nom, deux points d'arrêt dans le chef d'orchestre
- [ ] Plan validé, tests rouges puis verts
- [ ] Audit du diff fait, failles traitées
- [ ] Diff des tests relu, PR décrite, CI verte

PIÈGES À OBSERVER

- Oublier les points d'arrêt : l'agent enchaîne tout, et vous découvrez 20 fichiers modifiés.
- Un nom d'agent mal orthographié : la délégation échoue ou part vers un agent générique.
- Orchestrer une petite tâche : chaque sous-agent consomme des crédits. On n'orchestre pas une faute de frappe.
- Des `handoffs` copiés d'un ancien tutoriel : ignorés par le harnais Copilot.

BONUS

Lancez la revue en parallèle : « Utilise Architecte, Testeur et Auditeur comme sous-agents, en parallèle, pour relire ma PR, puis fais la synthèse. »

COINCÉ ?

- Le chef d'orchestre code tout lui-même ? Vérifiez qu'il a l'outil `agent` et que le corps nomme les agents exactement comme leur champ `name`.
- Plus que 10 minutes ? Réduisez la feature à un seul lot, mais allez jusqu'à l'audit et la PR.

ANNEXE A · CHEAT SHEET

A Session, Chat et raccourcis

Les quatre réglages d'une session

Liste	Ce que c'est	Valeurs
Session · harnais	qui fait tourner la boucle de l'agent : outils, contexte, permissions	Copilot (défaut), Local (déprécié), Claude, Codex, Cloud
Rôle	ce qu'on demande à l'agent d'être	Ask (répond), Plan (planifie, ne modifie rien), Agent (agit), vos agents
Modèle	le cerveau : change la qualité et le coût, **jamais les droits**	Auto, modèles rapides, modèles de raisonnement
Environnement	où s'exécutent les commandes, où vont les modifications	votre poste, une copie de travail, le cloud

Réglages de la journée : session **Copilot**, permissions **manuelles**, modèle **fixé** pendant les mesures (TP 1), Auto ensuite.

Quel rôle pour quel besoin ?

Besoin	Rôle
Comprendre, expliquer, diagnostiquer sans rien toucher	Ask
Découper une tâche multi-fichiers en lots vérifiables	Plan
Exécuter **un** lot validé : fichiers, commandes, tests	Agent
Un rôle récurrent avec ses outils et ses interdits	votre agent personnalisé
Une ligne évidente, un motif répétitif	suggestions inline (Tab)

Raccourcis (macOS : Cmd et Option)

Raccourci	Action
`Ctrl+Alt+I`	vue Chat
`Ctrl+I`	chat en ligne : modifier la sélection sur place
`Ctrl+N` (dans la vue Chat)	nouvelle session : contexte propre
`Tab` / `Échap`	accepter / refuser la suggestion
`Ctrl+→`	accepter mot par mot
`Alt+]` `Alt+[`	suggestion suivante / précédente
`Alt+\`	déclencher une suggestion
`Ctrl+Maj+P` › « Chat: Open Customizations »	voir ce que la session charge

Les leviers du Chat

Levier	Sert à	Exemples
`#`	choisir ce que le modèle lit	`#file` `#selection` `#codebase` `#changes` `#problems` `#fetch`
`@`	choisir qui répond	`@vscode` `@terminal` `@github`
`/`	lancer une intention prête, ou **vos skills**	`/explain` `/fix` `/tests` `/doc` `/compact` `/init` `/nom-de-skill`

Peu de fichiers bien choisis valent mieux que tout le dépôt.

Ce que Copilot voit

- **Inline** : le code avant et après le curseur, des extraits des onglets ouverts. **Pas** vos fichiers d'instructions : montrez-lui un fichier exemplaire et un commentaire structuré.
- **Chat et agent** : prompt système et outils, vos instructions, vos références `#`, l'historique de la session (erreurs comprises), les résultats d'outils… et votre message.

Réflexes de session

- Une tâche, une session. Session longue et confuse ? `/compact`, ou repartir d'une session neuve avec le plan validé.
- Sous chaque réponse : dépliez **Références**. Vos instructions et skills doivent y figurer.
- Permissions manuelles : lisez chaque commande avant de l'approuver (`rm`, `curl … | sh`, `git push --force`, installation de paquet).
- Un sous-agent apparaît replié dans le Chat avec son nom : dépliez-le pour voir ce qu'il a lu et rendu.
- Un raccourci ne répond pas ? `Ctrl+K` `Ctrl+S` pour le retrouver ou le réattribuer.
- Les noms de commandes, de rôles et de modèles évoluent chaque mois : la documentation de votre version de VS Code fait foi.

ANNEXE B · CHEAT SHEET

B Prompts avancés : la boîte à outils

Le prompt structuré : cinq composants

Composant	Question	Exemple SalleLibre
Contexte	où, avec quoi ?	« Dans `src/domain`, types `Booking` de `#file:types.ts`. »
Tâche bornée	quoi, et seulement quoi ?	« Écris `overlaps(a, b)`, rien d'autre. »
Contraintes	quelles règles ?	« Intervalles `[début, fin[`, fonction pure, aucune dépendance. »
Exemples attendus	quels cas tranchent ?	« 10h–11h et 11h–12h → false ; 10h–11h et 10h30–11h30 → true. »
Critère de fin	quand est-ce fini ?	« Arrête-toi quand ces exemples sont des tests verts. »

Les exemples attendus sont la contrainte la plus forte : ils deviennent les tests.

Le multi-étapes : pas d'étape sans preuve

1. **Clarifier** : « Avant d'écrire, pose-moi les questions qui manquent. »
2. **Planifier** (rôle Plan) : lots de 1 à 3 fichiers, chacun avec le test qui le valide. Pas de code.
3. **Exécuter un lot** (rôle Agent) : « Lot 1 seulement. Arrête-toi quand ses tests passent. »
4. **Relire** : « Relis ton diff : oublis, cas limites, écarts au plan. » Puis commit.

Comparer deux prompts

- Même tâche, **même modèle**, session neuve, au moins 2 à 3 essais par version.
- Grille décidée **avant** les essais : critères binaires ou comptables (compile, tests verts, bornes, allers-retours).
- Un seul changement entre deux versions.

Le commentaire structuré (inline)

Signature typée + JSDoc / docstring / Javadoc / XML doc : entrée, sortie, règle, exemple (dont un cas refusé), exception. Le meilleur prompt pour l'inline… et il reste dans le code.

Prompts prêts à copier

```
Clarifier
Je veux [objectif]. Avant d'écrire du code, pose-moi au plus
5 questions sur ce qui est ambigu (règles, bornes, erreurs).
```

```
Plan en lots (rôle Plan)
Propose un plan pour [fonctionnalité] en lots de 1 à 3 fichiers.
Pour chaque lot : fichiers, interfaces, test qui le valide.
Respecte PLAN.md. Aucun code.
```

```
Un lot (rôle Agent)
Exécute uniquement le lot [n] du plan validé. Tests d'abord.
Arrête-toi dès que ses tests passent et résume le diff.
```

```
Revue du diff
Relis #changes comme un relecteur exigeant : bugs, cas limites,
écarts à PLAN.md et aux instructions, tests affaiblis.
Liste les problèmes, ne corrige rien.
```

```
Générer à l'image de l'existant
Crée RoomService exactement sur le modèle de
#file:BookingService,
tests compris. Demande avant de t'écarter du modèle.
```

```
Comparer trois implémentations
Propose trois implémentations de [fonction] aux compromis différents.
Pour chacune : complexité, avantages, risques.
Ne choisis pas : présente un tableau.
```

```
Rétro-prompt
Voici l'implémentation attendue : #selection.
Quel prompt l'aurait produite du premier coup ?
```

ANNEXE C · CHEAT SHEET

Les fichiers qui guident Copilot

Quel fichier pour quel besoin ?

Fichier	Rôle	Chargé
`.github/copilot-instructions.md`	les règles de la maison	à chaque requête
`AGENTS.md` (racine)	idem, format partagé entre outils	à chaque requête
`.github/instructions/*.instructions.md`	règles pour certains fichiers	si `applyTo` correspond
`~/.copilot/copilot-instructions.md`	vos préférences personnelles	dans tous vos projets
`.github/skills/<nom>/SKILL.md`	une méthode, avec ses fichiers	si la demande s'y prête, ou `/nom`
`.github/agents/<nom>.agent.md`	un rôle : consignes, outils, modèle	choisi ou sollicité
`.vscode/mcp.json`	des outils externes pour l'agent	quand l'agent en a besoin

Le harnais Copilot lit ces fichiers à la même place dans VS Code, le CLI et l'app. **Ignorés** par le harnais Copilot : les prompt files `.prompt.md` (convertissez-les en skills) et les `handoffs` des agents.

Instructions ciblées

```
---
applyTo: "tests/**"
description: Conventions des tests de SalleLibre
---
- Un test paramétré par règle métier.
- Une horloge fixe, jamais l'heure réelle.
```

Motifs : `tests/**`, `src/api/**,src/services/**`, `**/*.py`. `**` = autant écrire `copilot-instructions.md`.

Skill

```
---
name: matrice-de-tests          # = nom du dossier
description: Quand l'utiliser, avec les mots des demandes.
disable-model-invocation: true  # facultatif : seulement par /nom
---
La méthode, en étapes. Les fichiers : [modele](./modele.md)
```

Chargement progressif : nom + description toujours lus ; le corps si la demande correspond ; les fichiers cités si nécessaire.

Agent personnalisé

```
---
name: Auditeur          # nom affiché et délégué
description: Ce qu'il fait, quand le choisir,
  ce qu'il ne fait jamais.
tools: ['read', 'search']  # le minimum
skills: ['owasp-revue']    # facultatif
model: …                   # facultatif
user-invocable: false      # masqué de la liste
disable-model-invocation: true  # jamais délégué
---
Mission, méthode, interdits vérifiables.
```

Écrire un bon fichier

- **Court** : chaque ligne se paie à chaque requête. 40 lignes au plus pour les instructions de dépôt.
- **Impératif et vérifiable** : « ConflictError → 409 », pas « gère bien les erreurs ».
- **Une seule source** : jamais `copilot-instructions.md` et `AGENTS.md` qui se contredisent.
- **Rien de secret** : ces fichiers partent dans les requêtes.
- **Versionné et relu en PR**, comme du code. Une règle d'équipe va dans le dépôt, pas dans vos préférences.

Ma règle n'est pas appliquée : diagnostic

Symptôme	À vérifier
Absent des Références	chemin et nom exacts ; Chat: Open Customizations
`.instructions.md` jamais chargé	motif `applyTo` (`test/**` ≠ `tests/**`)
Skill jamais chargée	description trop vague ; appel direct par `/nom`
Agent absent de la liste	extension `.agent.md`, dossier, `user-invocable`
Délégation ratée	nom exact (casse) ; outil `agent` dans `tools`
Règle appliquée une fois sur deux	règle floue, ou contredite par un autre fichier
Rien ne marche dans l'inline	normal : l'inline ne lit pas les instructions

D

ANNEXE D · CHEAT SHEET

Agents et orchestration

Les outils et leur risque

Outil	Permet de	Risque
`read`	lire les fichiers	faible
`search`	chercher dans le code	faible
`todo`	tenir une liste de tâches	nul
`edit`	créer et modifier des fichiers	moyen
`agent`	solliciter d'autres agents (sous-agents)	moyen
`execute`	lancer des commandes (terminal)	élevé
`web`	lire des pages Internet	élevé

`execute` + `web` : une page piégée peut dicter une commande (injection de prompt). Jamais `'*'` par confort.

Appeler un agent

- **Le choisir** dans la liste des rôles du Chat.
- **Le solliciter comme sous-agent** : « Utilise l'agent Auditeur comme sous-agent pour auditer src/api. » Le sous-agent travaille dans **son propre contexte** et ne renvoie que son résultat.
- **Un orchestrateur** : un agent avec l'outil `agent`, dont le corps nomme les équipiers **exactement** comme leur `name`.

Quatre schémas d'orchestration

Schéma	Quand	Points d'arrêt
Relais par lots	nouvelle feature	le plan, puis le diff
Éventail parallèle	revue d'une PR	la synthèse
Boucle courte	bug en production	la PR
Garde-fou	intégration sensible	le blocage, l'arbitrage

Six règles d'or

1. Un agent = un rôle, une description qui dit quand l'appeler.
2. Les outils minimaux : lecture seule dès que possible.
3. Des interdits vérifiables (« ne modifie jamais tests/ »).
4. Deux validations humaines : le plan, le diff.
5. Le diff des tests se relit avant celui du code.
6. Chaque sous-agent coûte : on n'orchestre pas une faute de frappe.

Les trois agents fournis pour le TP 6

```
---
name: Architecte
description: Découpe une demande de SalleLibre en lots
  vérifiables. Ne code jamais.
tools: ['read', 'search']
---
Tu es l'Architecte de SalleLibre.
- Lis PLAN.md et le code concerné.
- Propose des lots de 1 à 3 fichiers,
  chacun avec le test qui le valide.
- Signale tout écart à PLAN.md.
- Tu ne modifies aucun fichier.
```

```
---
name: Testeur
description: Écrit les tests d'un lot de SalleLibre à
partir de la
  matrice validée. Ne touche jamais au code de production.
tools: ['read', 'search', 'edit', 'execute']
skills: ['matrice-de-tests']
---
Tu es le Testeur de SalleLibre.
- Matrice validée d'abord, puis les tests.
- Un nouveau test doit d'abord être ROUGE.
- Tu ne modifies que tests/, jamais src/.
- Tu ne supprimes ni n'affaiblis jamais un test.
```

```
---
name: Documentaliste
description: Met à jour openapi.yaml et README.md de
SalleLibre
  à partir du code et des tests.
tools: ['read', 'search', 'edit']
---
Tu es le Documentaliste de SalleLibre.
- Pars du code et des tests, pas du plan.
- Mets à jour openapi.yaml et README.md :
  exemples de requête et codes d'erreur.
- Tu ne modifies aucun autre fichier.
```

Le Refactoreur (TP 4), l'Auditeur (TP 5) et le Chef d'orchestre (TP 6) sont dans les pages de leur TP.

ANNEXE E · CHEAT SHEET

E Tests et refactoring

La matrice de cas, avant tout test

Nominal	Limites	Négatifs	Sécurité
réservation valide → 201	10h–11h puis 11h–12h : accepté	chevauchement → 409	sans jeton → 401
recherche avec résultats	créneau à la minute de fermeture	`fin ≤ début` → 400	réservation d'autrui → 403
annulation par le propriétaire	capacité exactement égale	salle inexistante → 404	membre qui crée une salle → 403

Au moins deux cas par colonne. Les « ? » se tranchent **avant** la génération.

Un bon test généré

- Un nom qui dit la règle : « refuse un créneau qui chevauche une réservation existante ».
- **Arrange / Act / Assert**, une raison d'échouer.
- Une horloge **fixe**, jamais la vraie date du jour.
- Paramétré : une règle, une table de cas (`test.each` , `@pytest.mark.parametrize` , `@ParameterizedTest` , `[Theory]`).
- Vérifie un **état** (le résultat) plutôt qu'une interaction (quelle méthode a été appelée).
- Doubles aux **frontières** que vous possédez (persistance, horloge, notifications), jamais le domaine.

Cinq pièges des tests générés

Piège	Symptôme
Nom vague	« test 1 », « should work »
Vraie date	test vert aujourd'hui, rouge demain
Assertion faible	`toBeDefined()` , `assert result`
Sur-mock	le test vérifie le mock, pas le code
Test désactivé	`skip` , assertion commentée, « pour passer la suite »

Couverture et mutation

- **Couverture** = ce qui a été **exécuté**, pas ce qui a été vérifié.
- **Mutation** = changez `<` en `<=` : si aucun test ne casse, vos tests ne protègent pas cette ligne.

Refactoring : un principe par demande

```
[Symptôme] BookingService valide, calcule les prix et
envoie des mails.
[Principe] Responsabilité unique.
[Cible] Extrais l'envoi de mail dans NotificationService,
injecté.
[Contraintes] Comportement identique, aucun test modifié,
lance les tests avant et après, un seul refactoring.
```

Symptôme	Principe	Geste
une classe fait trois métiers	S	extraire une classe
`switch` sur le type de salle	O	polymorphisme, stratégie
sous-classe qui lève « non supporté »	L	revoir la hiérarchie
interface fourre-tout	I	séparer les interfaces
`new` du repository dans le service	D	injecter l'interface

La discipline

1. Tests verts **avant**.
2. Un refactoring, un diff relisible.
3. Tests verts **après**, aucun fichier de test modifié.
4. Un commit `refactor : …` . Le bug trouvé au passage : un commit `fix : …` séparé.

Optimiser

- **Mesurer d'abord** : un benchmark reproductible, des volumes réalistes.
- Donnez la mesure et la cible : « 120 ms pour 500 salles et 10 000 réservations, cible < 20 ms, même résultat. »
- Mesurer après. Le plus malin n'est pas toujours le plus rapide.

Moderniser du legacy

1. Faire expliquer (`/explain` , glossaire, diagramme) sans rien toucher.
2. Figer le comportement : tests de caractérisation, y compris les bizarreries.
3. Refactorer par petits pas sous ces tests.
4. Corriger les bugs ensuite, chacun avec son test.

F

ANNEXE F · CHEAT SHEET

Sécurité, dépendances et revue

OWASP Top 10 : 2025, appliqué au code généré

Catégorie	Ce que le code généré oublie
A01 · Contrôle d'accès défaillant	le propriétaire n'est jamais vérifié (IDOR)
A02 · Mauvaise configuration	CORS ouvert, mode debug, en-têtes absents
A03 · Chaîne d'approvisionnement	paquets hallucinés, vulnérables, non épinglés
A04 · Défaillances cryptographiques	MD5, secret faible, aléa prévisible
A05 · Injection	SQL ou commande par concaténation
A06 · Conception non sécurisée	ni limite de débit, ni quota
A07 · Authentification défaillante	JWT non vérifié, sans expiration
A08 · Intégrité des logiciels et données	désérialisation, actions CI non épinglées
A09 · Journalisation et alerte	secrets loggés, aucun audit
A10 · Conditions exceptionnelles	erreur avalée, trace renvoyée au client

Prompt d'audit (rôle attaquant)

```
Tu es un attaquant. Audite #file:bookings.routes avec la grille
OWASP Top 10 : 2025. Pour chaque faille : ligne, criticité, la requête HTTP qui l'exploite, le correctif. Ne corrige rien.
```

Prompt de correctif ciblé

```
Faille : DELETE /bookings/:id supprime la réservation d'autrui.
1. Écris d'abord le test qui l'exploite (doit être rouge).
2. Corrige : propriétaire ou admin, sinon 403.
3. Ne touche à rien d'autre. Tests verts.
```

Dépendances proposées par l'IA

- Le paquet **existe-t-il** sur le registre officiel ? Âge, mainteneurs, téléchargements, dépôt source.
- Un nom inconnu et plausible : risque de **slopsquatting** (un attaquant publie le nom halluciné).
- Version **actuelle**, pas celle de la date d'entraînement du modèle.
- Préférez ce que l'équipe a déjà validé. Lockfile commité, audit en CI.
- Ne demandez pas au modèle si son paquet est fiable : vérifiez hors du modèle.

Jamais dans un prompt

Mot de passe, jeton, clé d'API, chaîne de connexion · donnée personnelle ou client · code sous NDA hors du périmètre autorisé · contenu de fichiers `.env`.

Checklist de revue d'un diff généré

- [ ] Fait ce qui est demandé, **seulement** ce qui est demandé
- [ ] Le diff des **tests** relu en premier : rien de supprimé, sauté ou affaibli
- [ ] Cas limites : vide, null, bornes, doublons, fuseaux horaires
- [ ] Fonctions et API qui existent vraiment, dans la bonne version
- [ ] Contrôle d'accès sur chaque route qui lit ou modifie une ressource
- [ ] Entrées validées, requêtes paramétrées, réponses en DTO
- [ ] Aucun secret, aucune donnée personnelle dans le code ou les logs
- [ ] Dépendance ajoutée : vérifiée et acceptée
- [ ] Licence : pas de gros bloc recopié d'un dépôt public
- [ ] Taille relisible : au-delà de 400 lignes, redécouper
- [ ] Je peux l'expliquer ligne à ligne

Pull request d'un agent

Une issue avec critères d'acceptation → l'agent travaille → description générée **puis relue** → la CI prouve → revue Copilot → **un humain décide**, et ce n'est pas celui qui a lancé l'agent.

CI/CD générée : quatre vérifications

Permissions en lecture seule par défaut · actions épinglées par SHA · installation reproductible · portes de qualité bloquantes (tests, analyse, secrets).